Code Explanation

Testing Data Loader Module

The provided code is a test suite for a data loader module, written using the Pytest framework. It tests the functionality of loading different types of data (transactions, customer data, and store data) into Spark DataFrames.

Overview of the Code
The code defines several test fixtures to create sample data and a Spark session. It then uses these fixtures to test the data loading functions. The tests cover loading data from different file formats (CSV, Parquet, and Delta).

Step 1: Creating a Spark Session Fixture
This step creates a Pytest fixture to establish a Spark session for testing purposes. The fixture is defined with a module scope, meaning it will be created once per test module.

```
@pytest.fixture(scope="module")
def spark():
    """Create a Spark session for testing."""
    return (SparkSession.builder
            .master("local[2]")
            .appName("TestDataLoader")
            .config("spark.sql.warehouse.dir", tempfile.mkdtemp())
            .config("spark.sql.extensions", "io.delta.sql.DeltaSpark
SessionExtension")
            .config("spark.sql.catalog.spark_catalog", "org.apache.s
park.sql.delta.catalog.DeltaCatalog")
            .getOrCreate())
```

Step 2: Creating Sample Data Fixtures
This step defines fixtures to create sample data for transactions, customers, and stores. Each fixture creates a Spark DataFrame with predefined data and schema.

```
@pytest.fixture(scope="module")
def sample_transaction_data(spark):
    """Create sample transaction data for testing."""
    data = [
        ("t1", "c1", datetime.now(), 100.0, "Grocery", "s1", "Credit
", "false"),
        ("t2", "c2", datetime.now(), 50.0, "Electronics", "s2", "Deb
it", "true"),
        ("t3", "c1", datetime.now(), 75.0, "Clothing", "s3", "Cash",
 "false")
    ]

    schema = StructType([
        StructField("transaction_id", StringType(), False),
        StructField("customer_id", StringType(), False),
        StructField("transaction_date", TimestampType(), False),
        StructField("amount", DoubleType(), False),
        StructField("category", StringType(), True),
        StructField("store_id", StringType(), True),
```

```
        StructField("payment_method", StringType(), True),
        StructField("is_online", StringType(), True)
    ])

    return spark.createDataFrame(data, schema)
```

Step 3: Testing the get_spark_session Function
This step tests the `get_spark_session` function to ensure it returns a valid SparkSession.

```
def test_get_spark_session():
    """Test that get_spark_session returns a valid SparkSession."""
    spark = get_spark_session()
    assert spark is not None
    assert isinstance(spark, SparkSession)
```

Step 4: Testing Data Loading Functions
This step tests the data loading functions (`load_transactions`, `load_customer_data`, and `load_store_data`) by creating temporary files with sample data, loading the data using the respective functions, and verifying the loaded data.

```
def test_load_transactions_csv(spark, sample_transaction_data, tmpdi
r):
    """Test loading transactions from CSV."""
    # Create a temporary CSV file with sample data
    csv_path = os.path.join(tmpdir, "transactions")
    sample_transaction_data.write.csv(csv_path, header=True)

    # Load the data using the function
    loaded_df = load_transactions(spark, csv_path, "csv")

    # Verify the data was loaded correctly
    assert loaded_df.count() == 3
    assert "transaction_id" in loaded_df.columns
    assert "amount" in loaded_df.columns
```

Step 5: Testing Customer and Store Data Loading
This step is similar to Step 4 but focuses on loading customer and store data from Parquet and Delta files, respectively.

```
def test_load_customer_data(spark, sample_customer_data, tmpdir):
    """Test loading customer data."""
    # Create a temporary parquet file with sample data
    parquet_path = os.path.join(tmpdir, "customers")
    sample_customer_data.write.parquet(parquet_path)

    # Load the data using the function
    loaded_df = load_customer_data(spark, parquet_path, "parquet")

    # Verify the data was loaded correctly
    assert loaded_df.count() == 3
```

```
    assert "customer_id" in loaded_df.columns
    assert "loyalty_tier" in loaded_df.columns

def test_load_store_data(spark, sample_store_data, tmpdir):
    """Test loading store data."""
    # Create a temporary delta table with sample data
    delta_path = os.path.join(tmpdir, "stores")
    sample_store_data.write.format("delta").save(delta_path)

    # Load the data using the function
    loaded_df = load_store_data(spark, delta_path, "delta")

    # Verify the data was loaded correctly
    assert loaded_df.count() == 3
    assert "store_id" in loaded_df.columns
    assert "region" in loaded_df.columns
```